Enable your tokens in CCIP (Burn & Mint): Register from Safe multisig using Hardhat

This tutorial will guide you through enabling your tokens in CCIP using Hardhat and Safe Multisig smart accounts. You will learn how to deploy tokens and set up *Burn & Mint* token pools using a 2-of-3 multi-signature Safe. After that, you will register the tokens in CCIP and configure them using multisig transactions without needing manual intervention. Finally, you will test the **Burn & Mint** token handling mechanism, where tokens are burned on the source blockchain, and an equivalent amount is minted on the destination blockchain.

Introduction to Smart Accounts and Safe Multisig

Introduction

A **smart account** (also known as a smart contract account) leverages the programmability of smart contracts to extend their functionality and improve their security compared to externally owned accounts (EOAs). Smart accounts are controlled by one or multiple EOAs or other smart accounts, and all transactions must be initiated by one of these controllers.

Some common features of smart accounts include:

- **Multi-signature schemes**: Require multiple approvals for a transaction to be executed, enhancing security.
- **Transaction batching**: Combine multiple actions into a single transaction, reducing costs and improving efficiency.

DevHub

Safe is one of the most trusted implementations of a smart account, offering a robust multi-signature mechanism. In this tutorial, we will use a Safe Multisig account, specifically a 2-of-3 multi-signature setup, where **two** out of **three** owners must approve a transaction to execute it. This setup ensures enhanced security and decentralized control over the assets.

The **Protocol Kit** from Safe allows developers to interact with Safe, smart accounts through a TypeScript interface. This kit can be used to create new Safe accounts, update configurations, propose transactions, and execute them, making it an ideal choice for blockchain projects.

> **SAFE TRANSACTION SERVICE API**
>
> In this tutorial, we will use **BASE Sepolia** and **Ethereum Sepolia** testnets. This is because Safe relies on the **Safe Transaction Service API**. The Safe Transaction Service offers a REST API to track transactions sent via the Safe Smart Account. It also provides endpoints to send transactions, collect signatures off-chain, and inform the owners about pending transactions to be broadcasted on-chain. The **Safe SDK** provides the **API Kit**, a TypeScript client for the Safe Transaction Service API, which is currently only supported on specific testnets such as BASE Sepolia and Ethereum Sepolia. For more information, see the Safe Transaction Service supported networks.

How Safe Multisig Works

Before we proceed, let's take a moment to understand how the multisig transaction process works with a Safe smart account.

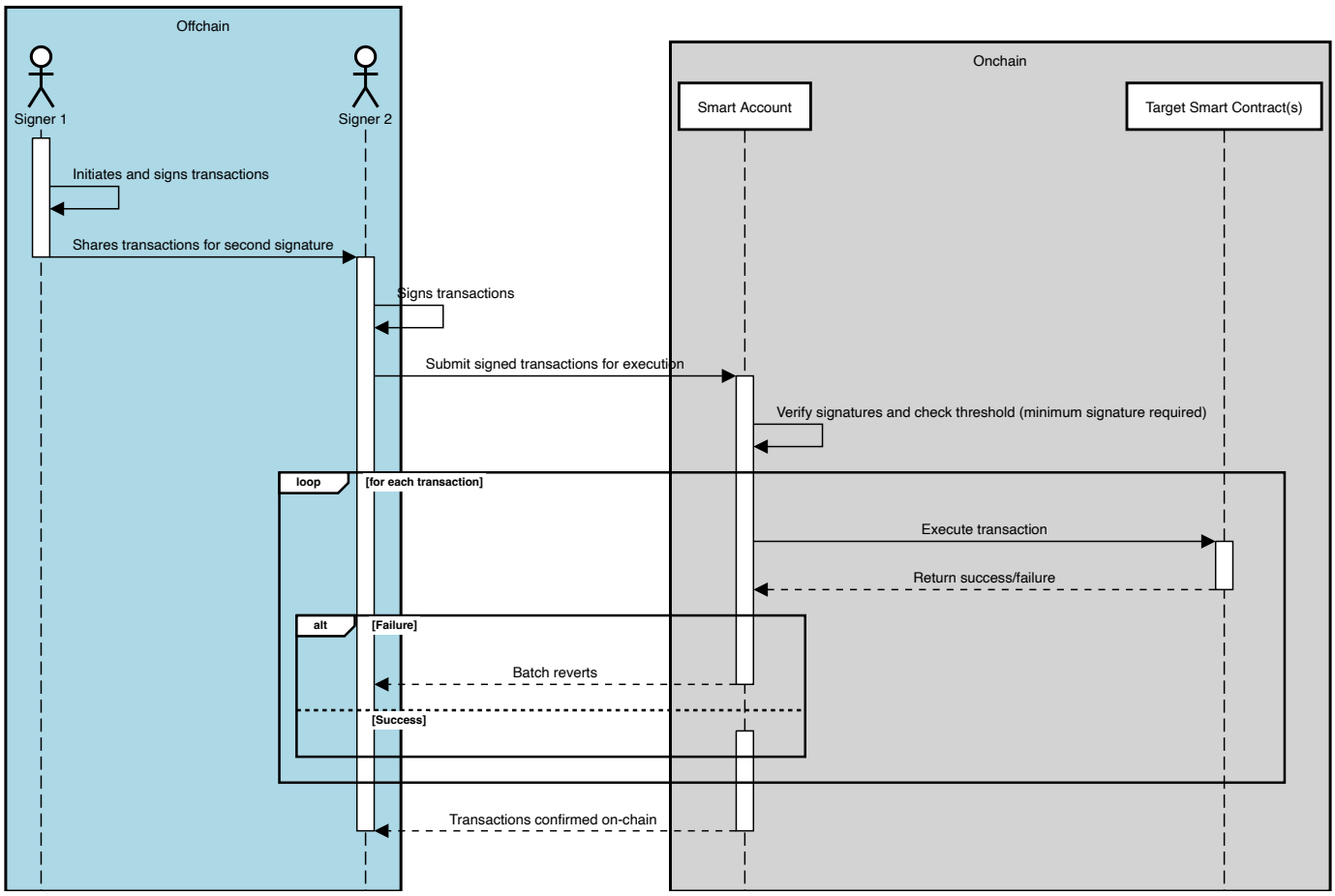
In this tutorial, we'll use a **2-of-3 multi-signature** setup as an example, where at least two out of the three Smart Account Owners must approve each transaction. Depending on your project's needs, this multisig process is valid for any scheme (e.g., 3-of-5, 4-of-7). In Safe smart accounts, the **Smart Account Owners** are responsible for approving actions. Transactions are signed off-chain by the required number of owners, which improves efficiency and reduces gas costs. Once the necessary threshold is met, the signed transactions are submitted to the blockchain for execution. The Safe smart account (a smart contract) verifies the signatures and enforces the required number of approvals before

DevHub

ensures that no single owner can act alone.

The steps for enabling your tokens in CCIP follow the same flow as the previous tutorials that used externally owned accounts (EOA). The key difference here is that for sensitive actions like token administrator registration or token pool configuration, we assume your project uses a Safe multisig. Therefore, multiple signatures are required off-chain; in some cases, a batch of transactions will be submitted to save on gas costs.

The following sequence diagram illustrates the multisig transaction flow in a Safe smart account, from off-chain signature collection to on-chain execution:



In this diagram, the process is as follows:

Ask AI

1. **Signer 1** initiates the batch of transactions (one or multiple transactions) and signs off-chain.

submitted for execution.

4. The **Smart Account** verifies the signatures and checks that the signature threshold has been met.
5. The **Smart Account** then executes each transaction in the batch, interacting with the Target Smart Contract(s).
6. If any transaction fails, the entire batch is reverted. If all transactions succeed, they are confirmed on-chain.

By following this process, we maintain the security of multisig transactions while improving efficiency through the off-chain signature collection and gas savings from transaction batching.

Steps covered in this tutorial

We will cover the following key steps:

1. **Creating a Safe Account**: You will create a 2-of-3 multi-signature Safe that will serve as the owner of the token and token pool contracts. This Safe will also manage administrative tasks, such as configuring token pools and registering as the token admin in the token admin registry.
2. **Deploying Tokens**: You will deploy your `BurnMintERC20` tokens on the Ethereum Sepolia and BASE Sepolia testnets and transfer ownership to the Safe account.
3. **Deploying Token Pools**: Once your tokens are deployed, you will deploy `BurnMintTokenPool` token pools on Ethereum Sepolia and BASE Sepolia. The Safe account will own each token pool.
4. **Claiming and Accepting the Admin Role**: This is a two-step process that will be managed using the Safe multi-signature account. It involves creating and signing multiple meta-transactions off-chain before executing them on-chain to register Safe as the token admin and accept the admin role for managing the tokens and token pools.

    1. You will call the `RegistryModuleOwnerCustom` contract's `registerAdminViaGetCCIPAdmin` function to register the Safe as the token admin.

function to complete the registration process.

Meta-transactions are used here to batch these two actions, allowing both steps to be executed efficiently. The meta-transactions are created off-chain and signed by each of the two required Safe owners. This off-chain signing process reduces gas costs and enhances security, as the transactions are only broadcasted to the blockchain once all required signatures are collected.

5. **Linking Tokens to Pools**: You will use the Safe account to call the `TokenAdminRegistry` contract's `setPool` function to associate each token with its respective token pool.
6. **Configuring Token Pools**: You will configure each token pool by setting cross-chain transfer parameters, such as token pool rate limits and enabled destination chains, using multisig transactions through the Safe account.
7. **Granting Mint and Burn Roles**: You will grant the mint and burn roles to the token pools on each linked token using the Safe account. These roles are required for the token pools to mint and burn tokens during cross-chain transfers.
8. **Minting Tokens**: You will mint tokens on Ethereum Sepolia. These tokens will later be used to test cross-chain transfers to BASE Sepolia.
9. **Transferring Tokens**: Finally, you will transfer tokens from Ethereum Sepolia to BASE Sepolia using CCIP. You can pay CCIP fees using either LINK tokens or native gas tokens.

By the end of this tutorial, you will have successfully deployed, registered, configured, and enabled your tokens and token pools for use in CCIP. All are managed securely through a multi-signature Safe account.

Before You Begin

1. Make sure you have Node.js v22.10.0 or above installed. If not, **install Node.js v22.10.0**:

   Download Node.js v22.10.0 if you don't have it installed. Optionally, you can use the nvm package to switch between Node.js versions:

DevHub

Verify that the correct version of Node.js is installed:

```
node -v
```

Example output:

```
$ node -v
v22.15.0
```

> **WHY NODE.JS V22?**
>
> Since the migration to Hardhat 3.x, Node.js v22.10.0 or above is required for compatibility. Using an older version may lead to unexpected issues during development.

2. **Clone the repository and navigate to the project directory:**

```
git clone https://github.com/smartcontractkit/smart-contract-examples.git
cd smart-contract-examples/ccip/cct/hardhat
```

3. **Install dependencies for the project:**

```
npm install
```

4. **Compile the project:**

```
npm run compile
```

5. **Encrypt your environment variables for higher security:**

Ask AI

The project uses @chainlink/env-enc to encrypt your environment variables at rest. Follow the steps below to configure your environment securely:

DevHub

```
npx env-enc set-pw
```

2. Set up a `.env.enc` file with the necessary variables for Ethereum Sepolia and BASE Sepolia testnets. Use the following command to add the variables:

```
npx env-enc set
```

Variables to configure:

- `ETHEREUM_SEPOLIA_RPC_URL`: A URL for the *Ethereum Sepolia* testnet. You can get a personal endpoint from services like Alchemy or Infura.
- `BASE_SEPOLIA_RPC_URL`: A URL for the *BASE Sepolia* testnet. You can sign up for a personal endpoint from Alchemy or Infura.
- `PRIVATE_KEY`: The private key for the first signer of the Safe multisig account. If you use MetaMask, you can follow this guide to export your private key. **Note:** This key is used to create and sign the transaction of the first signer.
- `PRIVATE_KEY_2`: The private key for the second signer of the Safe multisig account. If you use MetaMask, you can follow this guide to export your private key. **Note:** This key is used to create and sign the transaction of the second signer.
- `ETHERSCAN_API_KEY`: An API key from Etherscan to verify your contracts. You can obtain one from Etherscan.

6. **Fund the EOA linked to the first private key with LINK and native gas tokens**:
Make sure your EOA has enough LINK and native gas tokens on Ethereum Sepolia and BASE Sepolia to cover transaction fees. You can use the Chainlink faucets to get testnet tokens. **Important clarifications**:
   - Off-chain signatures are collected for this tutorial. The first EOA is responsible for sending the transactions to the Safe smart account, meaning only the first EOA requires enough native gas tokens for these transactions.

DevHub

EOA has sufficient LINK tokens to cover these fees. If a different EOA were to initiate the CCIP transfer, that EOA would need to hold enough LINK tokens.

Tutorial

EXPLORE THE CODE

All Hardhat tasks used in this tutorial are located in the `tasks/safe-multisig` directory of the repository. Each task is thoroughly commented and directly linked to a key step in the tutorial, making the code self-explanatory. Read the code and comments to gain a deeper understanding of the process or explore the implementation details.

FUNDING AND SIGNATURE CLARIFICATIONS

In this tutorial, the first EOA is responsible for sending transactions involving the Safe smart account. Therefore, **only the first EOA must have sufficient native gas tokens to cover transaction fees** for all operations. It is also important to note that during multi-signature operations, **signatures will be collected from the second EOA off-chain** to fulfill the required number of signatures, but the first EOA will handle the on-chain transaction submissions.

USING DIFFERENT NETWORKS

This tutorial uses Ethereum Sepolia and BASE Sepolia by default. To test with other CCIP-supported networks that support Safe Transaction Service, see Add CCIP Networks for Cross-Chain Token Tutorials.

Deploy Safe Smart Accounts

In this step, you will deploy a Safe smart account on both Ethereum Sepolia and BASE Sepolia using the `deploySafe` task. The Safe smart account will serve as the multi-signature account, requiring approvals from multiple owners to authorize transactions. You can customize the number of owners and the required threshold for signatures.

Ask AI

Below is an explanation of the parameters used during deployment:

DevHub

`owners`	Ethereum addresses that will control the Safe smart account and authorize transactions.	N/A	Yes
`threshold`	The number of required signatures to authorize a transaction. This must be at least 1 and cannot exceed the number of owners provided.	`1`	Yes
`network`	The blockchain on which the Safe smart account will be deployed. Examples include `ethereumSepolia` for Ethereum Sepolia and `baseSepolia` for BASE Sepolia.	N/A	Yes

1. **Deploy a Safe on Ethereum Sepolia** (Replace `0xOwnerAddress1`, `0xOwnerAddress2`, and `0xOwnerAddress3` with your Ethereum addresses):

```
npx hardhat deploySafe --owners "0xOwnerAddress1,0xOwnerAddress2,0xOwnerAddress
```

Example output:

```
$ npx hardhat deploySafe --owners 0x8C244f0B2164E6A3BED74ab429B0ebd661Bb1fCA

✅ Tasks loaded from /tasks/index.ts
2025-10-22T15:02:27.024Z info: ⚙️  Deploying Safe multisig on ethereumSepol
2025-10-22T15:02:27.026Z info:    Owners (3): 0x8C244f0B2164E6A3BED74ab429B0
2025-10-22T15:02:27.026Z info:    Threshold: 2
2025-10-22T15:02:27.026Z info:    Initializing Safe Protocol Kit...
2025-10-22T15:02:27.026Z info:    Salt nonce: 0x3bd008a765832c18c35a3f31f49
2025-10-22T15:02:27.027Z info:    Deploying Safe contract...
2025-10-22T15:02:33.751Z info:    Predicted Safe address: 0xD87c0BCbB9df8C07
2025-10-22T15:02:36.827Z info:    Deploying Safe contract on-chain...
2025-10-22T15:02:44.545Z info:    Deployment transaction: 0xa766fe32019a84d6
2025-10-22T15:02:44.546Z info:    Waiting for confirmation...
2025-10-22T15:03:04.169Z info: ✅ Safe deployed successfully
2025-10-22T15:03:04.170Z info:    Safe address: 0xD87c0BCbB9df8C07De0ea9af14
2025-10-22T15:03:04.170Z info:    Network: ethereumSepolia
```

Ask AI

DevHub

```
npx hardhat deploySafe --owners "0xOwnerAddress1,0xOwnerAddress2,0xOwnerAddress
```

Example output:

```
$ npx hardhat deploySafe --owners 0x8C244f0B2164E6A3BED74ab429B0ebd661Bb1fCA

✅ Tasks loaded from /tasks/index.ts
2025-10-22T15:04:29.505Z info: ⚙️  Deploying Safe multisig on baseSepolia...
2025-10-22T15:04:29.507Z info:    Owners (3): 0x8C244f0B2164E6A3BED74ab429B0
2025-10-22T15:04:29.507Z info:    Threshold: 2
2025-10-22T15:04:29.507Z info:    Initializing Safe Protocol Kit...
2025-10-22T15:04:29.508Z info:    Salt nonce: 0x06b5e0404c2a7c2e7be856725e66
2025-10-22T15:04:29.508Z info:    Deploying Safe contract...
2025-10-22T15:04:37.618Z info:    Predicted Safe address: 0xA9E127DeFf4f46dC
2025-10-22T15:04:39.082Z info:    Deploying Safe contract on-chain...
2025-10-22T15:04:44.074Z info:    Deployment transaction: 0xaea379d168d27f30
2025-10-22T15:04:44.075Z info:    Waiting for confirmation...
2025-10-22T15:04:44.463Z info: ✅ Safe deployed successfully
2025-10-22T15:04:44.463Z info:    Safe address: 0xA9E127DeFf4f46dCA489349A15
2025-10-22T15:04:44.463Z info:    Network: baseSepolia
```

RECORD SAFE CONTRACT ADDRESSES

Make sure to record the addresses of the deployed Safe smart accounts. You will need these addresses to interact with the Safe smart accounts in the following steps.

Deploy Tokens

In this step, you will deploy a token on both Ethereum Sepolia and BASE Sepolia using the `deployTokenWithSafe` task, then transfer ownership of the token to the Safe multisig account. This ensures that the Safe smart account controls the token, requiring multiple signatures to authorize any future administrative actions.

Ask AI

DevHub

Parameter	Description	Default	Required
`safeaddress`	The address of the Safe smart account that will own the deployed token.	N/A	Yes
`name`	The full name of the token.	N/A	Yes
`symbol`	The shorthand symbol representing the token.	N/A	Yes
`decimals`	The number of decimal places the token supports (e.g., `18` means 1 token is represented as 1e18 smallest units).	`18`	No
`maxsupply`	The maximum supply of tokens. Set to `0` for unlimited supply.	`0`	No
`premint`	The amount of tokens to be minted to the owner at the time of deployment. If set to `0`, no tokens will be minted to the owner during deployment.	`0`	No
`verifycontract`	Flag to verify the contract on Etherscan or a similar blockchain explorer. Pass this flag to enable verification, omit to skip.	`false`	No
`network`	The blockchain on which the token will be deployed. Examples include `ethereumSepolia` for Ethereum Sepolia and `baseSepolia` for BASE Sepolia.	N/A	Yes

1. **Deploy a token on Ethereum Sepolia** (Replace `0xSafeAddress` with the address of the Safe smart account. You can also adapt the token name and symbol as needed):

```
npx hardhat deployTokenWithSafe \
  --name "BnM sak" \
  --symbol BnMsak \
  --decimals 18 \
  --maxsupply 0 \
  --safeaddress 0xD87c0BCbB9df8C07De0ea9af14296248E082c347 \
  --verifycontract \
  --network ethereumSepolia
```

Ask AI

Example output:

DevHub

```
2025-10-22T15:08:03.911Z info:    Token: BnM sak (BnMsak)
2025-10-22T15:08:03.911Z info:    Decimals: 18
2025-10-22T15:08:03.911Z info:    Max supply: 0
2025-10-22T15:08:03.911Z info:    Premint: 0
2025-10-22T15:08:03.911Z info:    Safe address: 0xD87c0BCbB9df8C07De0ea9af14
2025-10-22T15:08:03.911Z info:    Deploying contract...
2025-10-22T15:08:19.639Z info: ✅ Token deployed at: 0xdca6ab0a735be79fd6b58
2025-10-22T15:08:19.640Z info:    Waiting for 3 confirmation(s)...
2025-10-22T15:08:28.643Z info:    Verifying contract...

📤 Submitted source code for verification on Etherscan:

  @chainlink/contracts/src/v0.8/shared/token/ERC20/BurnMintERC20.sol:BurnMir
  Address: 0xdca6ab0a735be79fd6b5864a3b1e3f597310b5da

⏳ Waiting for verification result...


✅ Contract verified successfully on Etherscan!

  @chainlink/contracts/src/v0.8/shared/token/ERC20/BurnMintERC20.sol:BurnMir
  Explorer: https://sepolia.etherscan.io/address/0xdca6ab0a735be79fd6b5864a3
2025-10-22T15:08:43.278Z info:    ✅ Token contract verified successfully
2025-10-22T15:08:43.278Z info:    Transferring ownership of token to Safe at
2025-10-22T15:08:44.181Z info:    Granting DEFAULT_ADMIN_ROLE to Safe: 0xD87
2025-10-22T15:08:47.077Z info:    ⏳ Grant role tx: 0xf6176199bdbc3b638c75c1
2025-10-22T15:09:16.482Z info:    ✅ Safe granted DEFAULT_ADMIN_ROLE
2025-10-22T15:09:16.482Z info:    Setting CCIP admin to Safe...
2025-10-22T15:09:18.450Z info:    ⏳ Set CCIP admin tx: 0x205fae446501ebf6f1
2025-10-22T15:10:04.979Z info:    ✅ Safe set as CCIP admin
2025-10-22T15:10:04.979Z info:
✅ Token deployment and configuration complete!
2025-10-22T15:10:04.979Z info:    Token address: 0xdca6ab0a735be79fd6b5864a3
2025-10-22T15:10:04.979Z info:    Safe address: 0xD87c0BCbB9df8C07De0ea9af14
```

Ask AI

2. **Deploy a token on BASE Sepolia** (Replace `0xSafeAddress` with the address of the Safe smart account. You can also adapt the token name and symbol as needed):

DevHub

```
  --symbol BnMsak \
  --decimals 18 \
  --maxsupply 0 \
  --safeaddress 0xA9E127DeFf4f46dCA489349A15F7EB8A059Fb116 \
  --verifycontract \
  --network baseSepolia
```

Example output:

```
✅ Tasks loaded from /tasks/index.ts
2025-10-22T15:11:31.865Z info: ⚙️  Deploying BurnMintERC20 on baseSepolia...
2025-10-22T15:11:31.867Z info:    Token: BnM sak (BnMsak)
2025-10-22T15:11:31.867Z info:    Decimals: 18
2025-10-22T15:11:31.867Z info:    Max supply: 0
2025-10-22T15:11:31.867Z info:    Premint: 0
2025-10-22T15:11:31.867Z info:    Safe address: 0xA9E127DeFf4f46dCA489349A15
2025-10-22T15:11:31.867Z info:    Deploying contract...
2025-10-22T15:11:39.696Z info: ✅ Token deployed at: 0x7074d32876ed00946d15e
2025-10-22T15:11:39.696Z info:    Waiting for 2 confirmation(s)...
2025-10-22T15:11:45.698Z info:    Verifying contract...

📤 Submitted source code for verification on Basescan:

   @chainlink/contracts/src/v0.8/shared/token/ERC20/BurnMintERC20.sol:BurnMi
   Address: 0x7074d32876ed00946d15ea71991eeb86be09666e

⏳ Waiting for verification result...


✅ Contract verified successfully on Basescan!

   @chainlink/contracts/src/v0.8/shared/token/ERC20/BurnMintERC20.sol:BurnMi
   Explorer: https://sepolia.basescan.org/address/0x7074d32876ed00946d15ea71
2025-10-22T15:12:08.298Z info:    ✅ Token contract verified successfully
2025-10-22T15:12:08.299Z info:    Transferring ownership of token to Safe at
2025-10-22T15:12:08.776Z info:    Granting DEFAULT_ADMIN_ROLE to Safe: 0xA9E
2025-10-22T15:12:11.801Z info:    ⏳ Grant role tx: 0xf1f1c1e57f2299e900ba79
```

DevHub

```
2025-10-22T15:12:24.474Z info:    ✅ Safe set as CCIP admin
2025-10-22T15:12:24.474Z info:
✅ Token deployment and configuration complete!
2025-10-22T15:12:24.474Z info:    Token address: 0x7074d32876ed00946d15ea719
2025-10-22T15:12:24.474Z info:    Safe address: 0xA9E127DeFf4f46dCA489349A15
```

RECORD TOKEN CONTRACT ADDRESSES

Make sure to record the addresses of the deployed token contracts. You will need these addresses to interact with the tokens in the following steps.

Deploy Token Pools

UNDERSTAND TOKEN POOL REQUIREMENTS

Before deploying your token pools, make sure you understand the mandatory requirements for token pools and the gas limit restrictions. The `releaseOrMint` function and other operations (e.g., balance checks) must not exceed the **90,000** gas limit on the destination blockchain. Failure to meet these requirements can lead to manual execution. For more details, refer to the Common Requirements.

In this step, you will deploy a token pool on both Ethereum Sepolia and BASE Sepolia using the `deployTokenPoolWithSafe` task, then transfer ownership of the token pool to the Safe smart account. This ensures that the Safe smart account controls the token pool, providing a secure, multisig setup for managing the token pool operations.

Below is an explanation of the parameters used during deployment:

Parameter	Description	Default	Required
`tokenaddress`	The address of the token that the pool will manage.	N/A	Yes
`safeaddress`	The address of the Safe smart account that will own the token pool.	N/A	Yes

DevHub

`ecimals`	The number of decimals for the token on this chain.	`18`	No
`verifycontract`	Flag to verify the contract on Etherscan or a similar blockchain explorer. Pass this flag to enable verification, omit to skip.	`false`	No
`network`	The blockchain on which the token pool will be deployed. Examples include `ethereumSepolia` for Ethereum Sepolia and `baseSepolia` for BASE Sepolia.	N/A	Yes

1. **Deploy a Burn and Mint token pool on Ethereum Sepolia** (Replace `0xTokenAddress` and `0xSafeAddress` with the token address and Safe smart account address, respectively):

```
npx hardhat deployTokenPoolWithSafe \
  --tokenaddress 0xTokenAddress \
  --safeaddress 0xSafeAddress \
  --localtokendecimals 18 \
  --verifycontract \
  --network ethereumSepolia
```

Example output:

```
$ npx hardhat deployTokenPoolWithSafe \
    --tokenaddress 0xdca6ab0a735be79fd6b5864a3b1e3f597310b5da \
    --safeaddress 0xD87c0BCbB9df8C07De0ea9af14296248E082c347 \
    --localtokendecimals 18 \
    --verifycontract \
    --network ethereumSepolia

✅ Tasks loaded from /tasks/index.ts
2025-10-22T15:20:07.175Z info: ⚙️ Deploying BurnMintTokenPool on ethereumSep
2025-10-22T15:20:07.177Z info: Token: 0xdca6ab0a735be79fd6b5864a3b1e3f597310
2025-10-22T15:20:07.177Z info: Safe:  0xD87c0BCbB9df8C07De0ea9af14296248E082
2025-10-22T15:20:07.177Z info: Decimals: 18
2025-10-22T15:20:11.885Z info: ⏳ Deployment tx: 0x4a6ad26be32f80ad61486122c
2025-10-22T15:20:11.886Z info:    Waiting for 3 confirmation(s)...
2025-10-22T15:20:52.348Z info: ✅ TokenPool deployed at: 0x4133f9c5d62Ac1e15
```

Ask AI

DevHub

```
If you need to verify a partially verified contract, please use the --force f

Explorer: https://sepolia.etherscan.io/address/0x4133f9c5d62Ac1e155B379695c3
2025-10-22T15:20:56.007Z info: ✅ TokenPool contract verified successfully
2025-10-22T15:20:56.008Z info: ⚙️ Transferring ownership to Safe: 0xD87c0BCb
2025-10-22T15:21:17.968Z info: ✅ Ownership transferred to Safe at 0xD87c0BC
```

2. **Deploy a Burn and Mint token pool on BASE Sepolia** (Replace `0xTokenAddress` and `0xSafeAddress` with the token address and Safe smart account address, respectively):

```
npx hardhat deployTokenPoolWithSafe \
  --tokenaddress 0xTokenAddress \
  --safeaddress 0xSafeAddress \
  --localtokendecimals 18 \
  --verifycontract \
  --network baseSepolia
```

Example output:

```
$ npx hardhat deployTokenPoolWithSafe \
   --tokenaddress 0x7074d32876ed00946d15ea71991eeb86be09666e \
   --safeaddress 0xA9E127DeFf4f46dCA489349A15F7EB8A059Fb116 \
   --localtokendecimals 18 \
   --verifycontract \
   --network baseSepolia

✅ Tasks loaded from /tasks/index.ts
2025-10-22T15:22:16.815Z info: ⚙️ Deploying BurnMintTokenPool on baseSepolia
2025-10-22T15:22:16.819Z info: Token: 0x7074d32876ed00946d15ea71991eeb86be09
2025-10-22T15:22:16.820Z info: Safe:  0xA9E127DeFf4f46dCA489349A15F7EB8A059F
2025-10-22T15:22:16.820Z info: Decimals: 18
2025-10-22T15:22:20.541Z info: ⏳ Deployment tx: 0x1eb547756ae29576375cefd4c
2025-10-22T15:22:20.541Z info:    Waiting for 2 confirmation(s)...
2025-10-22T15:22:24.985Z info: ✅ TokenPool deployed at: 0x48AF36da7c6cFb23A
2025-10-22T15:22:24.986Z info: ⚙️ Verifying contract...
```

DevHub

```
If you need to verify a partially verified contract, please use the --force f

Explorer: https://sepolia.basescan.org/address/0x48AF36da7c6cFb23A9C16eAE297
2025-10-22T15:22:28.530Z info: ✅ TokenPool contract verified successfully
2025-10-22T15:22:28.531Z info: ⚙️ Transferring ownership to Safe: 0xA9E127De
2025-10-22T15:22:30.727Z info: ✅ Ownership transferred to Safe at 0xA9E127D
```

RECORD TOKEN POOL CONTRACT ADDRESSES

Make sure to record the addresses of the deployed token pool contracts. You will need these addresses to interact with the token pools in the following steps.

Accept Ownership of Token Pools

After deploying the token pools and transferring ownership to the Safe smart account, the Safe smart account must formally accept ownership of the token pools. This ensures that all administrative actions for the token pools will require multisig approval, ensuring a secure and decentralized management process.

The process will use the Safe smart account to sign the transaction off-chain, collect the required signatures from multiple owners, and then execute it on-chain.

Below is an explanation of the parameters used during this task:

Parameter	Description	Required
`contractaddress`	The address of the contract whose ownership the Safe smart account is accepting.	Yes
`safeaddress`	The address of the Safe smart account that will accept ownership of the contract.	Yes
`network`	The blockchain network where the transaction will be executed. Examples include `ethereumSepolia` for Ethereum Sepolia and `baseSepolia` for BASE Sepolia.	Yes

DevHub

address, respectively):

```
npx hardhat acceptOwnershipFromSafe --contractaddress 0xContractAddress --safeac
```

Example output:

```
$ npx hardhat acceptOwnershipFromSafe \
    --contractaddress 0x4133f9c5d62Ac1e155B379695c31752249E2Ad4f \
    --safeaddress 0xD87c0BCbB9df8C07De0ea9af14296248E082c347 \
    --network ethereumSepolia

✅ Tasks loaded from /tasks/index.ts
2025-10-22T15:24:16.608Z info: ⚙️ Accepting ownership of contract 0x4133f9c5
2025-10-22T15:24:17.379Z info: ✅ Contract exists at 0x4133f9c5d62Ac1e155B37
2025-10-22T15:24:17.770Z info: ⚙️ Current owner: 0x8C244f0B2164E6A3BED74ab42
2025-10-22T15:24:17.770Z info: ✅ Current owner is 0x8C244f0B2164E6A3BED74ab
2025-10-22T15:24:17.770Z info: ⚙️ Simulating acceptOwnership transaction...
2025-10-22T15:24:18.049Z info: ✅ Simulation successful - Safe can accept ow
2025-10-22T15:24:18.050Z info: ⚙️ Initializing Safe Protocol Kit for multisig
2025-10-22T15:24:25.938Z info: ✅ Safe transaction created
2025-10-22T15:24:26.973Z info: ✅ Signed by owner 1
2025-10-22T15:24:27.653Z info: ✅ Signed by owner 2
2025-10-22T15:24:27.653Z info: ✅ Transaction has 2 signature(s)
2025-10-22T15:24:27.653Z info: 🚀 Executing Safe transaction to accept owner
2025-10-22T15:24:34.033Z info: ⏳ Waiting 3 blocks for tx 0x16f729787430afc0
2025-10-22T15:24:38.841Z info: ✅ Ownership accepted successfully for 0x4133
```

2. **Accept ownership of the token pool on BASE Sepolia** (Replace `0xContractAddress` and `0xSafeAddress` with the token pool contract address and Safe smart account address, respectively):

```
npx hardhat acceptOwnershipFromSafe \
  --contractaddress 0x48AF36da7c6cFb23A9C16eAE29766975F33dF032 \
```

DevHub

Example output:

```
✅ Tasks loaded from /tasks/index.ts
2025-10-22T15:25:24.635Z info: ⚙️ Accepting ownership of contract 0x48AF36d
2025-10-22T15:25:26.236Z info: ✅ Contract exists at 0x48AF36da7c6cFb23A9C1
2025-10-22T15:25:26.510Z info: ⚙️ Current owner: 0x8C244f0B2164E6A3BED74ab4
2025-10-22T15:25:26.510Z info: ✅ Current owner is 0x8C244f0B2164E6A3BED74a
2025-10-22T15:25:26.511Z info: ⚙️ Simulating acceptOwnership transaction...
2025-10-22T15:25:26.806Z info: ✅ Simulation successful - Safe can accept o
2025-10-22T15:25:26.807Z info: ⚙️ Initializing Safe Protocol Kit for multisi
2025-10-22T15:25:30.875Z info: ✅ Safe transaction created
2025-10-22T15:25:31.874Z info: ✅ Signed by owner 1
2025-10-22T15:25:33.189Z info: ✅ Signed by owner 2
2025-10-22T15:25:33.189Z info: ✅ Transaction has 2 signature(s)
2025-10-22T15:25:33.190Z info: 🚀 Executing Safe transaction to accept owne
2025-10-22T15:25:41.053Z info: ⏳ Waiting 2 blocks for tx 0x759d9dd2691ade67
2025-10-22T15:25:41.455Z info: ✅ Ownership accepted successfully for 0x48AF
```

Claim and Accept Token Admin Role using Safe

In this step, you will use the `claimAndAcceptAdminRoleFromSafe` task to claim and accept the admin role for the deployed tokens in a single Ethereum transaction. By leveraging Safe's batching feature, we can efficiently combine the two operations—claiming the admin role and accepting the admin role—into one on-chain interaction. This reduces gas costs and improves efficiency.

The process will use the Safe smart account to sign the transaction off-chain, collect the required signatures from multiple owners, and then execute it on-chain.

Below is an explanation of the parameters used during this task:

Ask AI

DevHub

`ess`	claimed and accepted.	N/A	Yes
`safeaddress`	The address of the Safe smart account that will execute the transactions and become the token admin.	N/A	Yes
`network`	The blockchain on which the transaction will be executed. Examples include `ethereumSepolia` for Ethereum Sepolia and `baseSepolia` for BASE Sepolia.	N/A	Yes

1. **Claim and accept the admin role for the token on Ethereum Sepolia** (Replace `0xTokenAddress` and `0xSafeAddress` with the token address and Safe smart account address, respectively):

```
npx hardhat claimAndAcceptAdminRoleFromSafe --tokenaddress 0xTokenAddress --sa
```

Example output:

```
$ npx hardhat claimAndAcceptAdminRoleFromSafe --tokenaddress 0xdca6ab0a739bc79

✅ Tasks loaded from /tasks/index.ts
2025-10-22T15:28:38.782Z info: ⚙️ Connecting to token contract at 0xdca6ab0c
2025-10-22T15:28:39.465Z info: ⚙️ Current CCIP admin: 0xD87c0BCbB9df8C07De0e
2025-10-22T15:28:39.465Z info: ✅ CCIP admin matches Safe address - proceed
2025-10-22T15:28:40.496Z info: ⚙️ Prepared Safe meta-transactions for 0xdca6
2025-10-22T15:28:40.496Z info: ⚙️ Initializing Safe Protocol Kit for multisig
2025-10-22T15:28:43.729Z info: ✅ Safe transaction (claim + accept) created
2025-10-22T15:28:44.140Z info: ✅ Signed by owner 1
2025-10-22T15:28:44.545Z info: ✅ Signed by owner 2
2025-10-22T15:28:44.545Z info: ✅ Transaction has 2 signature(s)
2025-10-22T15:28:44.546Z info: 🚀 Executing Safe transaction (claim + accept
2025-10-22T15:28:49.649Z info: ⏳ Waiting 3 blocks for tx 0x9b91763c65df9c27
2025-10-22T15:29:03.911Z info: ✅ Admin role claimed and accepted for 0xdca6
```

Ask AI

2. **Claim and accept the admin role for the token on BASE Sepolia** (Replace `0xTokenAddress` and `0xSafeAddress` with the token address and Safe smart account address, respectively):

DevHub

Example output:

```
$ npx hardhat claimAndAcceptAdminRoleFromSafe --tokenaddress 0x7074d32876

✅ Tasks loaded from /tasks/index.ts
2025-10-22T15:29:46.121Z info: ⚙️ Connecting to token contract at 0x7074d32
2025-10-22T15:29:46.732Z info: ⚙️ Current CCIP admin: 0xA9E127DeFf4f46dCA48
2025-10-22T15:29:46.733Z info: ✅ CCIP admin matches Safe address - proceed
2025-10-22T15:29:49.539Z info: ⚙️ Prepared Safe meta-transactions for 0x7074
2025-10-22T15:29:49.540Z info: ⚙️ Initializing Safe Protocol Kit for multisi
2025-10-22T15:29:55.544Z info: ✅ Safe transaction (claim + accept) created
2025-10-22T15:29:56.133Z info: ✅ Signed by owner 1
2025-10-22T15:29:57.228Z info: ✅ Signed by owner 2
2025-10-22T15:29:57.228Z info: ✅ Transaction has 2 signature(s)
2025-10-22T15:29:57.229Z info: 🚀 Executing Safe transaction (claim + accept
2025-10-22T15:30:03.859Z info: ⏳ Waiting 2 blocks for tx 0xe5e1cc59d5b734dd
2025-10-22T15:30:04.219Z info: ✅ Admin role claimed and accepted for 0x7074
```

Grant Mint and Burn Roles using Safe

In this step, you will use the `grantMintBurnRoleFromSafe` task to grant mint and burn roles to both the token pool and the Safe smart account on Ethereum Sepolia and BASE Sepolia. The Safe smart account will handle the transaction to securely assign these roles, ensuring that multiple owners sign off on the operation. Granting mint and burn roles is essential to allow the token pool and the Safe account to manage token issuance and burning, and to prepare for future cross-chain transfers.

This process will grant:

- Mint and burn roles to the token pool for handling cross-chain operations.
- Mint and burn roles to the Safe smart account for minting tokens to EOAs for testing purposes.

Ask AI

Below is an explanation of the parameters used during this task:

DevHub

`ss`	roles will be granted.	Yes
`burnermint ers`	A comma-separated list of addresses (token pools and Safe smart account) to which mint and burn roles will be granted.	Yes
`safeaddres s`	The address of the Safe smart account that will execute the transaction to grant mint and burn roles.	Yes
`network`	The blockchain on which the mint and burn roles will be granted. Examples include `ethereumSepolia` for Ethereum Sepolia and `baseSepolia` for BASE Sepolia.	Yes

1. **Grant mint and burn roles on Ethereum Sepolia** (Replace `0xTokenAddress`, `0xPoolAddress`, and `0xSafeAddress` with the token address, token pool address, and Safe smart account address, respectively):

```
npx hardhat grantMintBurnRoleFromSafe \
  --tokenaddress 0xTokenAddress \
  --burnerminters 0xPoolAddress,0xSafeAddress \
  --safeaddress 0xSafeAddress \
  --network ethereumSepolia
```

Example output:

```
$ npx hardhat grantMintBurnRoleFromSafe --tokenaddress 0xdca6ab0a735be79f

✅ Tasks loaded from /tasks/index.ts
2025-10-22T15:37:10.201Z info: ⚙️ Connecting to token contract at 0xdca6ab0
2025-10-22T15:37:10.612Z info: ⚙️ Initializing Safe Protocol Kit for multisi
2025-10-22T15:37:13.743Z info: ⚙️ Setting up Safe transaction to grant roles
2025-10-22T15:37:14.323Z info: ✅ Safe transaction created
2025-10-22T15:37:16.990Z info: ✅ Signed by owner 1
2025-10-22T15:37:18.027Z info: ✅ Signed by owner 2
2025-10-22T15:37:18.027Z info: ✅ Transaction has 2 signature(s)
2025-10-22T15:37:18.027Z info: 🚀 Executing Safe transaction to grant roles.
```

DevHub

2. **Grant mint and burn roles on BASE Sepolia** (Replace `0xTokenAddress`, `0xPoolAddress`, and `0xSafeAddress` with the token address, token pool address, and Safe smart account address, respectively):

```
npx hardhat grantMintBurnRoleFromSafe \
  --tokenaddress 0xTokenAddress \
  --burnerminters 0xPoolAddress,0xSafeAddress \
  --safeaddress 0xSafeAddress \
  --network baseSepolia
```

Example output:

```
$ npx hardhat grantMintBurnRoleFromSafe --tokenaddress 0x7074d32876ed0094

✅ Tasks loaded from /tasks/index.ts
2025-10-22T15:38:21.300Z info: ⚙️ Connecting to token contract at 0x7074d328
2025-10-22T15:38:21.767Z info: ⚙️ Initializing Safe Protocol Kit for multisi
2025-10-22T15:38:26.148Z info: ⚙️ Setting up Safe transaction to grant roles
2025-10-22T15:38:26.451Z info: ✅ Safe transaction created
2025-10-22T15:38:27.971Z info: ✅ Signed by owner 1
2025-10-22T15:38:29.289Z info: ✅ Signed by owner 2
2025-10-22T15:38:29.289Z info: ✅ Transaction has 2 signature(s)
2025-10-22T15:38:29.289Z info: 🚀 Executing Safe transaction to grant roles.
2025-10-22T15:38:40.886Z info: ⏳ Waiting 2 blocks for tx 0xcd1317734142f9b9
2025-10-22T15:38:42.178Z info: ✅ Mint and burn roles granted successfully
```

Set Pool using Safe

In this step, you will use the `setPoolFromSafe` task to link a token to a token pool on both Ethereum Sepolia and BASE Sepolia. The Safe smart account will be used to execute the transaction, ensuring that the pool is set securely with multisig approval. Multiple owners will sign the transaction off-chain before it is executed on-chain.

DevHub

Parameter	Description	Required
`tokenaddress`	The address of the token for which the pool will be set.	Yes
`pooladdress`	The address of the token pool to be linked to the token.	Yes
`safeaddress`	The address of the Safe smart account that will execute the transaction.	Yes
`network`	The blockchain on which the transaction will be executed. Examples include `ethereumSepolia` for Ethereum Sepolia and `baseSepolia` for BASE Sepolia.	Yes

1. **Set the token pool on Ethereum Sepolia** (Replace `0xTokenAddress`, `0xPoolAddress`, and `0xSafeAddress` with the token address, token pool address, and Safe smart account address, respectively):

```
npx hardhat setPoolFromSafe --tokenaddress 0xTokenAddress --pooladdress 0xP
```

Example output:

```
$ npx hardhat setPoolFromSafe --tokenaddress 0xdca6ab0a735be79fd6b5864a3b

✅ Tasks loaded from /tasks/index.ts
2025-10-22T15:40:57.725Z info: ⚙️ Connecting to TokenAdminRegistry at 0x95F
2025-10-22T15:40:58.327Z info: ⚙️ Preparing to set pool for token 0xdca6ab0a7
2025-10-22T15:40:58.328Z info: ⚙️ Initializing Safe Protocol Kit for multisi
2025-10-22T15:41:01.865Z info: ✅ Safe transaction created
2025-10-22T15:41:02.576Z info: ✅ Signed by owner 1
2025-10-22T15:41:02.980Z info: ✅ Signed by owner 2
2025-10-22T15:41:02.980Z info: ✅ Transaction has 2 signature(s)
2025-10-22T15:41:02.980Z info: 🚀 Executing Safe transaction to set pool for
2025-10-22T15:41:12.075Z info: ⏳ Waiting 3 blocks for tx 0xd981df37f210b759
2025-10-22T15:41:25.444Z info: ✅ Pool set for token 0xdca6ab0a735be79fd6b586
```

DevHub

address, respectively):

```
npx hardhat setPoolFromSafe --tokenaddress 0xTokenAddress --pooladdress 0xP
```

Example output:

```
$ npx hardhat setPoolFromSafe --tokenaddress 0x7074d32876ed00946d15ea7199

✅ Tasks loaded from /tasks/index.ts
2025-10-22T15:42:57.749Z info: ⚙️ Connecting to TokenAdminRegistry at 0x736
2025-10-22T15:42:58.543Z info: ⚙️ Preparing to set pool for token 0x7074d3287
2025-10-22T15:42:58.544Z info: ⚙️ Initializing Safe Protocol Kit for multisi
2025-10-22T15:43:04.152Z info: ✅ Safe transaction created
2025-10-22T15:43:04.833Z info: ✅ Signed by owner 1
2025-10-22T15:43:05.425Z info: ✅ Signed by owner 2
2025-10-22T15:43:05.425Z info: ✅ Transaction has 2 signature(s)
2025-10-22T15:43:05.425Z info: 🚀 Executing Safe transaction to set pool for
2025-10-22T15:43:13.454Z info: ⏳ Waiting 2 blocks for tx 0xa3eed5c5a01581e5
2025-10-22T15:43:13.746Z info: ✅ Pool set for token 0x7074d32876ed00946d15ea
```

Configure Token Pools using Safe

In this step, you will use the `applyChainUpdatesFromSafe` task to configure a token pool for cross-chain interactions. By leveraging the Safe smart account, you can securely update the configuration of the token pool to support remote chains, including setting rate limits and linking it to remote pools and tokens.

The task handles complex cross-chain setups, allowing you to define rate limits for both inbound and outbound token transfers. The transaction is signed by the Safe owners off-chain and then executed on-chain, ensuring secure multi-signature control over the pool configuration.

Below is an explanation of the parameters used during this task:

DevHub

`remotechain`	The identifier of the remote blockchain (e.g., `ethereumSepolia` for Ethereum Sepolia or `baseSepolia` for BASE Sepolia).	N/A	Yes
`remotepooladdresses`	Comma-separated list of remote pool addresses.	N/A	Yes
`remotetokenaddress`	The address of the token on the remote chain.	N/A	Yes
`outboundratelimitenabled`	Flag to enable outbound rate limits for cross-chain transfers. Pass this flag to enable, omit to disable.	`false`	No
`outboundratelimitcapacity`	The maximum number of tokens that can be transferred outbound in a single burst (bucket capacity for the outbound rate limiter).	`0`	No
`outboundratelimitrate`	The rate at which tokens are refilled in the outbound bucket (tokens per second).	`0`	No
`inboundratelimitenabled`	Flag to enable inbound rate limits for cross-chain transfers. Pass this flag to enable, omit to disable.	`false`	No
`inboundratelimitcapacity`	The maximum number of tokens that can be transferred inbound in a single burst (bucket capacity for the inbound rate limiter).	`0`	No
`inboundratelimitrate`	The rate at which tokens are refilled in the inbound bucket (tokens per second).	`0`	No
`safeaddress`	The address of the Safe smart account that will execute the transaction to configure the pool.	N/A	Yes
`network`	The blockchain on which the pool is being configured. Examples include `ethereumSepolia` for Ethereum Sepolia and `baseSepolia` for BASE Sepolia.	N/A	Yes

1. **Configure the token pool on Ethereum Sepolia** (Replace `0xPoolAddress`, `0xRemotePoolAddress`, `0xRemoteTokenAddress`, and `0xSafeAddress` with the token pool address, remote token pool address, remote token address, and Safe smart account address, respectively):

Ask AI

DevHub

Example output:

```
$ npx hardhat applyChainUpdatesFromSafe --pooladdress 0x4133f9c5d62Ac1e15

✅ Tasks loaded from /tasks/index.ts
2025-10-22T15:51:43.618Z info: ⚙️ Applying chain updates for pool 0x4133f9c5
2025-10-22T15:51:44.419Z info: ⚙️ Initializing Safe Protocol Kit for multisi
2025-10-22T15:51:52.717Z info: ✅ Safe transaction created
2025-10-22T15:51:53.691Z info: ✅ Signed by owner 1
2025-10-22T15:51:54.656Z info: ✅ Signed by owner 2
2025-10-22T15:51:54.656Z info: ✅ Transaction has 2 signature(s)
2025-10-22T15:51:54.656Z info: 🚀 Executing Safe transaction for pool 0x4133
2025-10-22T15:52:02.815Z info: ⏳ Waiting 3 blocks for tx 0x532b168b2d987a6e
2025-10-22T15:52:16.503Z info: ✅ Pool configured successfully for baseSepol
```

2. **Configure the token pool on BASE Sepolia** (Replace `0xPoolAddress`, `0xRemotePoolAddress`, `0xRemoteTokenAddress`, and `0xSafeAddress` with the token pool address, remote token pool address, remote token address, and Safe smart account address, respectively):

```
npx hardhat applyChainUpdatesFromSafe --pooladdress 0xPoolAddress --remotechain
```

Example output:

```
$ npx hardhat applyChainUpdatesFromSafe --pooladdress 0x48AF36da7c6cFb23A

✅ Tasks loaded from /tasks/index.ts
2025-10-22T15:53:02.257Z info: ⚙️ Applying chain updates for pool 0x48AF36da
2025-10-22T15:53:03.055Z info: ⚙️ Initializing Safe Protocol Kit for multisi
2025-10-22T15:53:09.233Z info: ✅ Safe transaction created
2025-10-22T15:53:10.029Z info: ✅ Signed by owner 1
2025-10-22T15:53:11.255Z info: ✅ Signed by owner 2
2025-10-22T15:53:11.255Z info: ✅ Transaction has 2 signature(s)
2025-10-22T15:53:11.255Z info: 🚀 Executing Safe transaction for pool 0x48AF
```

Ask AI

DevHub

Mint Tokens to an EOA using Safe

In this step, you will use the `mintTokensFromSafe` task to mint tokens to an EOA on Ethereum Sepolia. This process uses a Safe smart account to securely manage the minting process, ensuring that the transaction is signed by multiple owners off-chain before being executed on-chain. These tokens will be used for transfers through CCIP from Ethereum Sepolia to BASE Sepolia.

Below is an explanation of the parameters used during this task:

Parameter	Description	Required
`tokenaddress`	The address of the token contract from which the tokens will be minted.	Yes
`amount`	The amount of tokens to mint for each recipient address.	Yes
`receiveraddresses`	A comma-separated list of recipient addresses (EOAs) that will receive the minted tokens.	Yes
`safeaddress`	The address of the Safe smart account that will execute the transaction to mint tokens.	Yes
`network`	The blockchain on which the minting transaction will be executed. For example, `ethereumSepolia` for Ethereum Sepolia.	Yes

1. **Mint tokens to an EOA on Ethereum Sepolia** (Replace `0xTokenAddress`, `0xSafeAddress`, and `0xReceiverAddress` with the token address, Safe smart account address, and recipient address, respectively):

```
npx hardhat mintTokensFromSafe \
  --tokenaddress 0xTokenAddress \
  --receiveraddresses 0xReceiverAddress \
  --amount 100000000000000000000 \
```

Ask AI

DevHub

Example output:

```
$ npx hardhat mintTokensFromSafe --tokenaddress 0xdca6ab0a735be79fd6b5864a5b1e

✅ Tasks loaded from /tasks/index.ts
2025-10-22T15:54:47.249Z info: ⚙️ Connecting to token contract at 0xdca6ab0c
2025-10-22T15:54:48.393Z info: ⚙️ Checking if Safe has MINTER_ROLE...
2025-10-22T15:54:48.393Z info: ✅ Safe has MINTER_ROLE - proceeding with min
2025-10-22T15:54:48.393Z info: ⚙️ Initializing Safe Protocol Kit for multisig
2025-10-22T15:54:54.934Z info: ⚙️ Preparing to mint 100000000000000000000 t
2025-10-22T15:54:55.526Z info: ✅ Safe transaction created
2025-10-22T15:54:56.553Z info: ✅ Signed by owner 1
2025-10-22T15:54:57.606Z info: ✅ Signed by owner 2
2025-10-22T15:54:57.606Z info: ✅ Transaction has 2 signature(s)
2025-10-22T15:54:57.607Z info: 🚀 Executing Safe transaction to mint tokens.
2025-10-22T15:55:05.307Z info: ⏳ Waiting 3 blocks for tx 0x9147027424f893cf
2025-10-22T15:55:14.261Z info: ✅ Tokens minted successfully via Safe multis
2025-10-22T15:55:15.033Z info: ℹ️ 0x8C244f0B2164E6A3BED74ab429B0ebd661Bb14CA
```

Transfer Tokens

In this step, you will use the `transferTokens` task to transfer tokens from Ethereum Sepolia to BASE Sepolia using CCIP. You have two options for paying CCIP fees: using LINK tokens or native gas tokens.

You will interact with the `IRouterClient` contract, specifically calling the `ccipSend()` function to initiate the token transfer.

Below is an explanation of the parameters used during the token transfer process:

Parameter	Description	Default	Required
`tokenaddress`	The address of the token being transferred.	N/A	Yes

DevHub

`destinationchain`	The blockchain to which the tokens will be transferred. Examples include `baseSepolia`, and `ethereumSepolia`.	N/A	Yes
`receiveraddress`	The address of the receiver on the destination blockchain.	N/A	Yes
`fee`	The type of fee used for the transfer, either `LINK` or `native`.	`LINK`	No
`network`	The blockchain on which the token transfer will be initiated. Examples include `baseSepolia`, and `ethereumSepolia`.	N/A	Yes

Pay fees in LINK

Call the CCIP Router to transfer tokens from Ethereum Sepolia to BASE Sepolia, paying the CCIP fees in LINK tokens. Replace the token address, amount, receiver address, and blockchain with the appropriate values:

```
npx hardhat transferTokens --tokenaddress 0xTokenAddress --amount 200000000000000
```

Example output:

```
$ npx hardhat transferTokens --tokenaddress 0xdca6ab0a735be79fd6b5864a3b1e3f597310

✅ Tasks loaded from /tasks/index.ts
2025-10-22T15:56:36.021Z info: 🚀 Transferring 2000000000000000000 tokens via
2025-10-22T15:56:36.022Z info:    Token: 0xdca6ab0a735be79fd6b5864a3b1e3f597310
2025-10-22T15:56:36.023Z info:    Receiver: 0xA028Cedc47485aB2F1230551E4f3a6871
2025-10-22T15:56:36.023Z info:    Fee token: LINK
2025-10-22T15:56:37.377Z info: 💰 Estimated fees: 12710869350886886
2025-10-22T15:56:37.702Z info: Approving 2000000000000000000 tokens for router
2025-10-22T15:56:40.548Z info:    Waiting for 3 confirmation(s)...
2025-10-22T15:57:16.204Z info: Approving 12710869350886886 LINK to router
2025-10-22T15:57:17.625Z info:    Waiting for 3 confirmation(s)...
2025-10-22T15:57:48.593Z info: 💰 Estimated CCIP fees: 12710869350886886
2025-10-22T15:57:48.594Z info: Simulating CCIP message...
```

DevHub

```
2025-10-22T15:58:27.246Z info: ✅ CCIP message sent successfully
2025-10-22T15:58:27.246Z info:    Transaction: 0xd365f959ff5be2a21a6cd742684575
2025-10-22T15:58:27.247Z info:    Check status: https://ccip.chain.link/tx/0xd3
2025-10-22T15:58:27.247Z info: 📋 Transfer Summary:
2025-10-22T15:58:27.247Z info:    Token: 0xdca6ab0a735be79fd6b5864a3b1e3f597310
2025-10-22T15:58:27.247Z info:    Amount: 2000000000000000000
2025-10-22T15:58:27.248Z info:    From: ethereumSepolia
2025-10-22T15:58:27.248Z info:    To: baseSepolia
2025-10-22T15:58:27.248Z info:    Receiver: 0xA028Cedc47485aB2F1230551E4f3a6871
2025-10-22T15:58:27.248Z info:    Fee paid in: LINK
```

You can check the status of the message on the Chainlink CCIP Explorer by visiting the provided URL. In this example, the message ID is `0x706c9057d25c69c9e1191a5a86b07a2156ef78bc7eaff6334c02dad5e905a9fb`.

Pay fees in native gas tokens

Call the CCIP Router to transfer tokens from Ethereum Sepolia to BASE Sepolia, paying the CCIP fees in native gas tokens. Replace the token address, amount, receiver address, and blockchain with the appropriate values:

```
npx hardhat transferTokens --tokenaddress 0xTokenAddress --amount 200000000000000
```

Example output:

```
$ npx hardhat transferTokens --tokenaddress 0xdca6ab0a735be79fd6b5864a3b1e3f597310

✅ Tasks loaded from /tasks/index.ts
2025-10-22T15:59:17.942Z info: 🚀 Transferring 2000000000000000000 tokens via
2025-10-22T15:59:17.943Z info:    Token: 0xdca6ab0a735be79fd6b5864a3b1e3f597310
2025-10-22T15:59:17.944Z info:    Receiver: 0xA028Cedc47485aB2F1230551E4f3a6871
2025-10-22T15:59:17.944Z info:    Fee token: native
2025-10-22T15:59:19.091Z info: 💰 Estimated fees: 64134211874037
2025-10-22T15:59:19.464Z info: Approving 2000000000000000000 tokens for router
```

DevHub

```
2025-10-22T15:59:49.354Z info: Sending CCIP message...
2025-10-22T15:59:50.740Z info: ⏳ CCIP message tx: 0x4796f9d7176c8bd90a320e373b
2025-10-22T15:59:50.740Z info:    Waiting for 3 confirmation(s)...
2025-10-22T16:00:26.196Z info: ✅ CCIP message sent successfully
2025-10-22T16:00:26.196Z info:    Transaction: 0x4796f9d7176c8bd90a320e373b0d73
2025-10-22T16:00:26.197Z info:    Check status: https://ccip.chain.link/tx/0x47
2025-10-22T16:00:26.197Z info: 📋 Transfer Summary:
2025-10-22T16:00:26.197Z info:    Token: 0xdca6ab0a735be79fd6b5864a3b1e3f597310
2025-10-22T16:00:26.197Z info:    Amount: 2000000000000000000
2025-10-22T16:00:26.197Z info:    From: ethereumSepolia
2025-10-22T16:00:26.197Z info:    To: baseSepolia
2025-10-22T16:00:26.197Z info:    Receiver: 0xA028Cedc47485aB2F1230551E4f3a6871
2025-10-22T16:00:26.197Z info:    Fee paid in: native
```

You can check the status of the message on the Chainlink CCIP Explorer by visiting the provided URL.

⚠ **EDUCATIONAL EXAMPLE DISCLAIMER**

This page includes an educational example to use a Chainlink system, product, or service and is provided to demonstrate how to interact with Chainlink's systems, products, and services to integrate them into your own. This template is provided "AS IS" and "AS AVAILABLE" without warranties of any kind, it has not been audited, and it may be missing key checks or error handling to make the usage of the system, product or service more clear. Do not use the code in this example in a production environment without completing your own audits and application of best practices. Neither Chainlink Labs, the Chainlink Foundation, nor Chainlink node operators are responsible for unintended outputs that are generated due to errors in code.

DevHub

Subscribe now

Developers

Developer Resources

Builder Quick Links

LINK Token Contracts

Data Feeds

Data Streams

VRF

Automation

Functions

CCIP

Solutions

Overview

DeFi

Chainlink VRF

Ask AI

Community

Chainlink Hackathon

DevHub

Events

Become an advocate

Code of conduct

Chainlink

Ecosystem

Press

Blog

Team

Brand assets

FAQs

Contact

Security

Support

Press inquiries

Social

Twitter

YouTube

Discord

Telegram

WeChat

Ask AI

DevHub